Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет «МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчёт по лабораторной работе № 3

по дисциплине «Технологии программирования»

на тему «Порождающие паттерны проектирования»

Выполнил:студент группы БИВТ-25-4-7Иванов Антон Сергеевич

Проверил:ассистентГласов А. В.

Москва, 2026

Цель и содержание работы

Цель работы — изучить порождающие паттерны проектирования на практике. По заданию требовалось реализовать Singleton, Factory Method, Abstract Factory и Builder на выбранном языке программирования и загрузить исходный код в удалённый репозиторий.

Работу выполнил на Python в PyCharm. Для каждого паттерна создал отдельный файл. Примеры взял из лекции и адаптировал с Java на Python, сохранив основные классы и методы.

Ход выполнения работы

1. Singleton (Одиночка)

В файле singleton.py создал класс Singleton. Поле _instance хранит ссылку на экземпляр, а метод __new__() при первом вызове создаёт объект и при последующих обращениях возвращает его снова. Для получения экземпляра добавил статический метод getInstance().

```python
class Singleton:  3 usages
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    @staticmethod  2 usages
    def getInstance():
        return Singleton()

    def someMethod(self):  1 usage
        print("Singleton is working")

if __name__ == "__main__":
    first = Singleton.getInstance()
    second = Singleton.getInstance()

    print("Same instance:", first is second)
    first.someMethod()
```

Рисунок 1 — Реализация паттерна Singleton

В конце программы дважды вызвал getInstance() и сравнил результаты оператором is. Значение True показало, что обе переменные ссылаются на один объект. Метод someMethod() также выполнился.

```
Same instance: True
Singleton is working

Process finished with exit code 0
```

Рисунок 2 — Результат запуска singleton.py

2. Factory Method (Фабричный метод)

В файле factory_method.py создал абстрактный класс Logger с методом log(message). От него наследуются FileLogger и ConsoleLogger. Как и в примере лекции, оба класса выводят сообщения в консоль, но с разными подписями.

```python
from abc import ABC, abstractmethod

class Logger(ABC):  2 usages
    @abstractmethod  1 usage (1 dynamic)
    def log(self, message):
        pass

class FileLogger(Logger):  1 usage
    def log(self, message):  1 usage (1 dynamic)
        print("Logging to file:", message)

class ConsoleLogger(Logger):  1 usage
    def log(self, message):  1 usage (1 dynamic)
        print("Logging to console:", message)
```

Рисунок 3 — Интерфейс Logger и конкретные виды логгеров

Затем создал абстрактный класс LoggerFactory. Метод createLogger() оставил абстрактным, а в logMessage() вызывается создание логгера и передача ему сообщения. В классах FileLoggerFactory и ConsoleLoggerFactory метод createLogger() возвращает разные объекты.

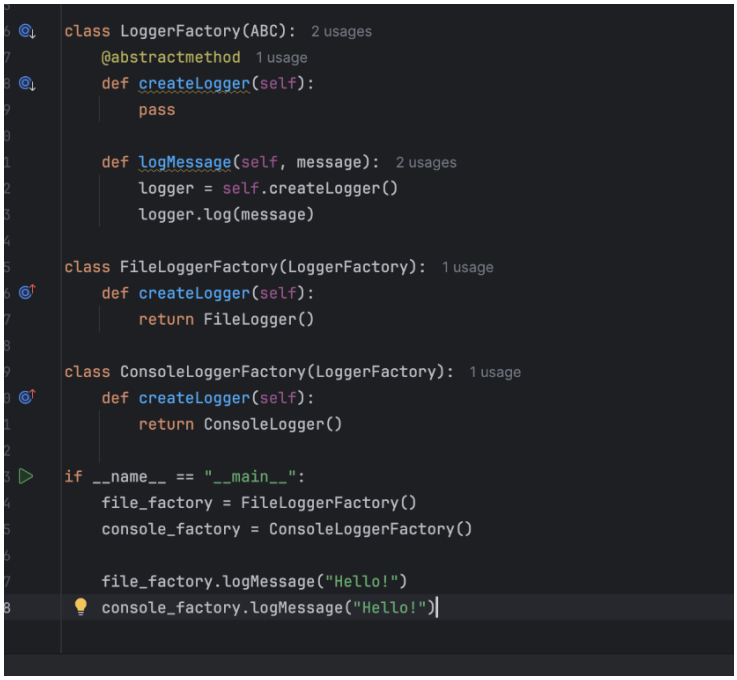


Рисунок 4 — Фабричный метод и конкретные фабрики

В примере использования создал две фабрики и передал им сообщение Hello!. Вывод подтвердил, что каждая фабрика использует нужный класс логгера.



Рисунок 5 — Результат запуска factory_method.py

3. Abstract Factory (Абстрактная фабрика)

Для примера с графическим интерфейсом создал два абстрактных продукта — Button и Checkbox, каждый с методом paint(). Конкретные классы WindowsButton, MacButton, WindowsCheckbox и MacCheckbox выводят сообщение о созданном элементе.



```python
from abc import ABC, abstractmethod

class Button(ABC):  2 usages
    @abstractmethod  2 usages (2 dynamic)
    def paint(self):
        pass

class WindowsButton(Button):  1 usage
    def paint(self):  2 usages (2 dynamic)
        print("You have created a Windows button.")

class MacButton(Button):  1 usage
    def paint(self):  2 usages (2 dynamic)
        print("You have created a Mac button.")

class Checkbox(ABC):  2 usages
    @abstractmethod  2 usages (2 dynamic)
    def paint(self):
        pass

class WindowsCheckbox(Checkbox):  1 usage
    def paint(self):  2 usages (2 dynamic)
        print("You have created a Windows checkbox.")

class MacCheckbox(Checkbox):  1 usage
    def paint(self):  2 usages (2 dynamic)
        print("You have created a Mac checkbox.")
```

Рисунок 6 — Кнопки и флажки для Windows и macOS

В интерфейсе GUIFactory объявил два метода: createButton() и createCheckbox(). WindowsFactory возвращает элементы Windows, а MacFactory — элементы Mac. Поэтому каждую фабрику можно использовать для получения согласованного набора элементов.

```python
class GUIFactory(ABC):  2 usages
    @abstractmethod  1 usage (1 dynamic)
    def createButton(self):
        pass

    @abstractmethod  1 usage (1 dynamic)
    def createCheckbox(self):
        pass

class WindowsFactory(GUIFactory):  1 usage
    def createButton(self):  1 usage (1 dynamic)
        return WindowsButton()

    def createCheckbox(self):  1 usage (1 dynamic)
        return WindowsCheckbox()

class MacFactory(GUIFactory):  1 usage
    def createButton(self):  1 usage (1 dynamic)
        return MacButton()

    def createCheckbox(self):  1 usage (1 dynamic)
        return MacCheckbox()

class Application:  2 usages
    def __init__(self, factory :{createButton, createCheckbox} ):
        self.button = factory.createButton()
        self.checkbox = factory.createCheckbox()
```

Рисунок 7 — Абстрактная фабрика и её реализации

Класс Application принимает фабрику в конструкторе и через неё создаёт кнопку и флажок. Чтобы сменить набор элементов, достаточно передать другую фабрику — сам класс Application менять не требуется.

```python
        self.button = factory.createButton()
        self.checkbox = factory.createCheckbox()

    def paint(self):  4 usages (2 dynamic)
        self.button.paint()
        self.checkbox.paint()

if __name__ == "__main__":
    app1 = Application(WindowsFactory())
    app1.paint()

    app2 = Application(MacFactory())
    app2.paint()
```

Рисунок 8 — Использование фабрик в классе Application

```
You have created a Windows button.
You have created a Windows checkbox.
You have created a Mac button.
You have created a Mac checkbox.

Process finished with exit code 0
```

Рисунок 9 — Результат запуска abstract_factory.py

4. Builder (Строитель)

В файле builder.py создал класс Pizza с полями dough, sauce и topping. Отдельные методы setDough(), setSauce() и setTopping() заполняют эти поля. Метод __str__() позволяет вывести состав готовой пиццы.

```python
from abc import ABC, abstractmethod

class Pizza:  1 usage
    def __init__(self):
        self.dough = None
        self.sauce = None
        self.topping = None

    def setDough(self, dough):  1 usage
        self.dough = dough

    def setSauce(self, sauce):  1 usage
        self.sauce = sauce

    def setTopping(self, topping):  1 usage
        self.topping = topping

    def __str__(self):
        return f"Pizza: {self.dough}, {self.sauce}, {self.topping}"

class PizzaBuilder(ABC):  1 usage
    @abstractmethod  1 usage (1 dynamic)
    def buildDough(self):
        pass
```

Рисунок 10 — Класс Pizza и интерфейс PizzaBuilder

Абстрактный строитель PizzaBuilder задаёт шаги приготовления. В HawaiianPizzaBuilder при создании объекта появляется пустая пицца, а методы buildDough(), buildSauce() и buildTopping() последовательно задают тесто, соус и начинку. getResult() возвращает собранный объект.

```python
class HawaiianPizzaBuilder(PizzaBuilder):  1 usage
    def __init__(self):
        self.pizza = Pizza()

    def buildDough(self):  1 usage (1 dynamic)
        self.pizza.setDough("cross")

    def buildSauce(self):  1 usage (1 dynamic)
        self.pizza.setSauce("mild")

    def buildTopping(self):  1 usage (1 dynamic)
        self.pizza.setTopping("ham+pineapple")

    def getResult(self):  1 usage
        return self.pizza
```

Рисунок 11 — Реализация HawaiianPizzaBuilder

Класс PizzaDirector получает строителя и вызывает три шага в методе constructPizza(). После выполнения этих шагов я получил пиццу через getResult() и вывел её состав на экран.

```python
class PizzaDirector:  1 usage
    def __init__(self, builder):
        self.builder = builder

    def constructPizza(self):  1 usage
        self.builder.buildDough()
        self.builder.buildSauce()
        self.builder.buildTopping()

if __name__ == "__main__":
    builder = HawaiianPizzaBuilder()
    director = PizzaDirector(builder)

    director.constructPizza()
    pizza = builder.getResult()

    print(pizza)
```

Рисунок 12 — Создание пиццы через директора

```
Pizza: cross, mild, ham+pineapple

Process finished with exit code 0
```

Рисунок 13 — Результат запуска builder.py

5. Размещение лабораторной на GitHub

Все четыре Python-файла сначала находились в отдельном проекте Lab_3. В локальном репозитории TP_Labs создал одноимённую папку и командой cp ../Lab_3/*.py Lab_3/ скопировал в неё только исходные файлы. После git add Lab_3/ проверил состав подготовленных изменений командой git status.

```
wxlchara@MacBook-Air-Anton-2 TP_Labs % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Lab_3/abstract_factory.py
        new file:   Lab_3/builder.py
        new file:   Lab_3/factory_method.py
        new file:   Lab_3/singleton.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .DS_Store
```

Рисунок 14 — Четыре файла Lab_3, подготовленные к коммиту

Создал коммит с сообщением Add Lab 3 - Creational Patterns. Первая отправка в GitHub не прошла, поскольку в удалённой ветке появились новые изменения. Командой git pull --rebase origin main получил их и повторно применил локальный коммит. После этого git push origin main прошёл успешно.

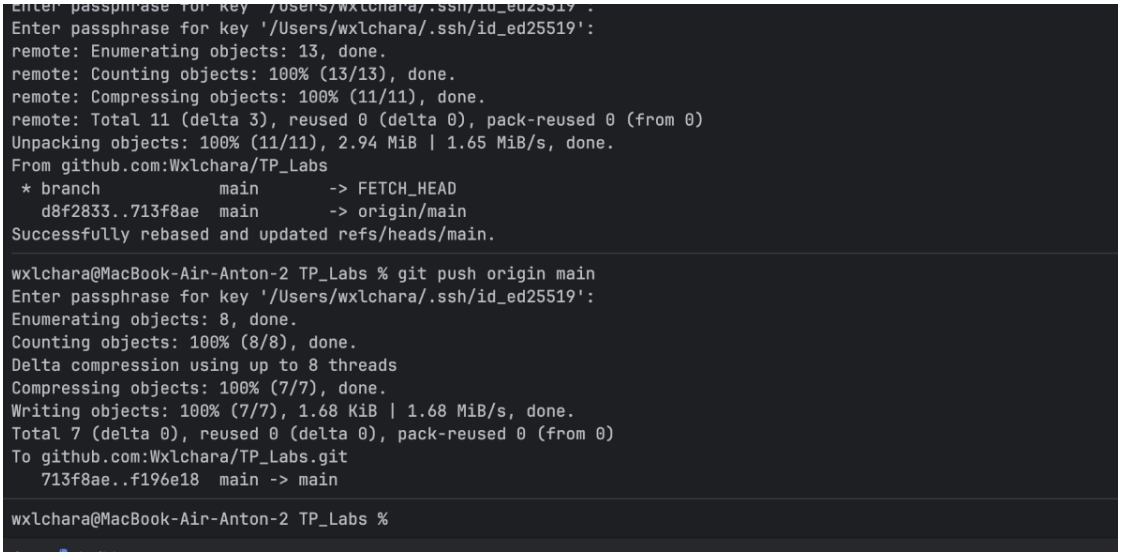


Рисунок 15 — Обновление ветки и успешная отправка на GitHub

После отправки git status показал, что ветка main синхронизирована с origin/main. В репозитории сохранились четыре файла в папке Lab_3. Служебный файл .DS_Store в коммит не включал.

Вывод

В ходе лабораторной работы реализовал четыре порождающих паттерна проектирования. Singleton позволяет обращаться к одному экземпляру класса, Factory Method передаёт выбор конкретного объекта подклассам, Abstract Factory создаёт семейства связанных объектов, а Builder разделяет создание объекта на последовательные этапы. Все примеры успешно запустил в PyCharm. Исходные файлы добавил в репозиторий TP_Labs и отправил на GitHub.